

DAY 22:
ASYNCHRONOUS
JAVASCRIPT –
PROMISES &
ASYNC/AWAIT
EXPLAINED!

JavaScript is single-threaded, meaning it executes code line by line. But what if we need to fetch data from an API, wait for a file to load, or run a time-consuming task? 🤔

This is where Asynchronous JavaScript comes in! 🚀

◆ What is Asynchronous JavaScript?

Asynchronous programming allows code execution without blocking other operations. It helps in:

- ✓ Fetching API data
- ✓ Handling user inputs smoothly
- ✓ Running background tasks

JavaScript provides multiple ways to handle async operations:

- Callbacks (Old approach 😞)
- Promises (Modern approach ✅)
- Async/Await (Even better! ✅✅)

◆ I. Callbacks (The Old Way) – ❌ Avoid This!

A callback function is passed as an argument to another function and runs after a task completes.

```
function fetchData(callback) {  
  setTimeout(() => {  
    console.log("Data fetched! ✅");  
    callback(); // Calling the callback  
    after fetching data  
  }, 2000);  
}
```

```
fetchData(() => {  
  console.log("Processing the data...");  
});
```

📌 Problem: Callback Hell (Too many nested callbacks make code unreadable!)

◆ 2. Promises – The Modern Approach

A Promise is an object that represents a future value. It has 3 states:

- ✓ Pending (Initial state)
- ✓ Resolved (Success )
- ✓ Rejected (Error )

✓ Example: Creating a Promise

```
const fetchData = new Promise((resolve,
reject) => {
  setTimeout(() => {
    let success = true; // Change to false to
test error
    if (success) {
      resolve("Data fetched successfully!
✓");
    } else {
      reject("Error fetching data ✗");
    }
  }, 2000);
});
// Handling the promise
fetchData
  .then(response => console.log(response)) //
Runs if resolved
  .catch(error => console.error(error)); //
Runs if rejected
```



Key Methods:

- `.then(response => { })` → Runs on success 
- `.catch(error => { })` → Handles errors 

◆ 3. Async/Await – The Best Approach!

Async/Await makes asynchronous code look synchronous & readable. It works with Promises.



Example: Using Async/Await

```
async function fetchUserData() {  
  try {  
    let response = await  
    fetch("https://jsonplaceholder.typicode  
    .com/users/1");
```

```
let data = await response.json();  
  console.log("User Data:", data);  
} catch (error) {  
  console.error("Error fetching user  
data:", error);  
}  
}  
fetchUserData();
```

Key Points:

- `async` makes a function return a Promise.
- `await` pauses execution until the Promise resolves.
- Use `try...catch` to handle errors.

◆ Promise vs Async/Await – Which One to Use?

Feature	Promises	Async/Await
Readability	😓 Nested <code>.then()</code> chains	✅ Looks synchronous
Error Handling	<code>.catch()</code> method	✅ <code>try...catch</code> block
Ease of Use	❌ Harder to debug	✅ Simpler & cleaner